

Training — the collection's shared cells

exp022 · 28 June 2026

Abstract

The collection's single training hub: every cell is trained once to a shared root, and the analysis notebooks load those weights instead of retraining (train-once / reuse-many). Status **draft** — nothing is trained to the gamma standard yet.

Methods

1. The compute cost

Each trial is surrogate-gradient backprop-through-time: 2000 timesteps ($T = 200$ ms at $\Delta t = 0.1$ ms) over a 1280-neuron conductance network. What makes it costly:

- **Fine timestep.** COBA synapses inject $g(V - E)$, stiffening the membrane so $\Delta t = 0.1$ ms — $10\times$ finer than a current-based model, and non-optional: the gamma rhythm lives there.
- **Sequential.** The 2000 steps are a dependency chain no hardware parallelises, so the job is **bandwidth-bound** — a GPU beats a CPU only $\approx 10\times$, not the $50\text{--}100\times$ of large matmuls.
- **Memory-heavy.** The backward pass stores every step's activations — ≈ 12 GB at batch 256.
- **Scale.** 87 cells $\times$ 50 epochs $\approx$ **185 A100-GPU-hours** ($\approx 49\text{M}$ sample-forwards) — ≈ 159 days on CPU, so every cell trains on a GPU.

2. Gold star trainings

87 cells across five families, each fixing its time constants, timestep, and MNIST fraction (re-split 80/20). The standard is $\tau_{\text{AMPA}} = 2$ ms, $\tau_{\text{GABA}} = 6$ ms (loop in gamma, ≈ 44 Hz); the sweeps each vary one axis — spike budget, τ_{GABA} , timestep (exp044), or init.

Family	τ_{GABA} (ms)	Δt (ms)	Epochs	MNIST	Cells	At spec
canonical (θ_{u} off)	6	0.1	50	all (70k)	6	0 / 6
θ_{u} sweep	6	0.1	50	10%	36	0 / 36
τ_{GABA} ladder	4.5 – 27	0.1	50	10%	18	0 / 18
Δt sweep	6	0.05 – 1.0	50	10%	15	0 / 15
init variants	6	0.1	50	10%	12	0 / 12
total					87	0 / 87

The **canonical** family (coba, ping at $\theta_{\text{u}} = \text{off}$) is the full-MNIST reference the sweeps are read against. *At spec* means trained to the gamma standard — **none yet**: the cells on disk from the original notebooks are at the old standard (2.9–5% MNIST, $\tau_{\text{GABA}} = 9$ ms), so the run trains all 87 fresh.

Choices behind the table:

- **50 epochs, halved from 100.** Accuracy plateaus by $\approx 15\text{--}20$ epochs (exp024); 50 keeps a ≈ 30 -epoch tail for the post-convergence *dynamics* the collection studies (rate drift, confidence inflation) while nearly halving the run.
- **3 seeds throughout.** Every cell — canonical and every sweep — trains three seeds (42, 43, 44), so each point on each frontier carries a cross-seed band and the headline effects (PING's rate attractor) read as robust, not one-run luck. The θ_{u} interior used to be single-seed; it no longer is.

- **Full MNIST vs 10%.** The canonical reference carries the headline numbers, so it sees all 70k; the sweeps need only the trend across their parameter, so 10% suffices and cuts their cost tenfold.

3. Compute options

Bandwidth is the main driver but not the whole story — the RTX 4090 below has half the A100’s bandwidth yet measured *faster*. Costs and wall-clocks cover the full 50-epoch registry; measured where the card was to hand, projected for the 5090:

Option	GPU (bandwidth)	Samples/s	Full-run cost	Wall-clock
Modal	A100 (2.0 TB/s)	74	≈ \$615	≈ half a day (fans out)
RunPod / Vast.ai	RTX 4090 fleet (1.0 TB/s)	100 (meas.)	≈ \$47 – 95	≈ 10 hr on ≈ 15 pods (≈ 5.7 d serial)
Cambridge Wilkes3	A100	74	≈ £102	offline through 2026
benjy (CUED, shared)	A6000 (768 GB/s)	26.5	£0	≈ 22 days
Owned workstation	RTX 5090 (1.8 TB/s)	≈ 150 (proj.)	≈ £4,700 once	≈ 3.8 days

The 4090 (≈ 100 samples/s, measured) beats both its bandwidth projection (≈ 37) and the A100, so the 5090 is scaled from it (≈ 1.5×). Wilkes3 is cheapest-and-fast but offline through 2026; benjy is one shared GPU forcing an older PyTorch.

The compute option we selected

The run fans out across RunPod **RTX 4090s**, split by stakes: the 6 heavy canonical cells (≈ 9.7 hr each) on **secure on-demand** — non-preemptible, so a mid-run death is rare — and the 81 light sweep cells (≈ 1 hr each) on cheaper **community** pods, where a lost cell costs an hour, not a day. Together ≈ \$67, against Modal’s ≈ \$615, and unlike the 5090 the 4090s actually provision. A pre-baked cu128 Docker image lets pods boot ready to train; a launcher buckets the 87 cells so the six canonical cells land on separate pods (pinning wall-clock to the ≈ 9.7 hr floor of one canonical cell), with a done-marker check keeping the run idempotent and resumable. Artifacts sync back to the shared root; the figures below build locally. At ≈ 15 pods (6 secure + 9 community) it finishes in ≈ 10 hours — more do not help, since one canonical cell cannot be split. The owned **5090** is the long-term answer for routine iteration.

4. Saved per cell

Each training writes one directory under the shared root (*src/artifacts/notebooks/training//*), with the same files every time — the contract the analysis notebooks read:

File	Holds
weights.pth · weights_final.pth	trained model state (all parameter tensors) — best-accuracy epoch and final epoch, which differ once the rate attractor drifts past the accuracy plateau (exp024).
config.json	full run config — model, lr, epochs, dt, t_ms, tau_gaba, θ_u, w_in, ei_strength, ei_ratio, readout_w_out_scale,

File	Holds
	max_samples, n_params, and provenance (git sha, run id, env hash).
metrics.json	summary + per-epoch history (below).
metrics.jsonl	the per-epoch records, one JSON per line (live streaming log).
test_predictions.json	predicted vs true label per test image.
output.log · run.sh	training stdout; the exact re-runnable train command.

Top-level fields in metrics.json: **mode**, **schema_version**, **model** (run mode, artifact-schema version, model name); **run_finished_at**; **config** (key fields incl. the sweep axes; full set in config.json); **init**, **end** (dynamics before/after — rate_e, rate_i, cv, act, contrast, f0_hz); **epochs** (per-epoch records, below); **best_acc**, **best_epoch**; **total_elapsed_s**; **perf** (device, torch, peak memory, epoch times, samples/sec).

Each per-epoch record — one entry in the *epochs* list, one line in metrics.jsonl:

Field	Holds
ep, acc	epoch number; test accuracy (%).
loss, test_loss	mean train / test cross-entropy loss.
rate_e, rate_i, test_rate_e, test_rate_i	mean E / I firing rate, train and test (Hz).
cv, act, contrast, f0_hz	ISI coefficient of variation; active hidden fraction; E-population rhythmicity (lobe–trough contrast in [0,1)) and gamma frequency (Hz, from the autocorrelogram lobe lag; null when no rhythm resolves).
lobe_lag_ms, trough_lag_ms, iei_mode_lag_ms, lobe_to_trough	raw rhythmicity scalars behind contrast/f0 — autocorrelogram lobe and trough lags, inter-event-interval mode lag, unbounded lobe/trough ratio.
lr, grad_norm	learning rate; clipped global gradient norm (epoch mean).
grad_norms, grad_ratios, weight_norms	per-weight-type gradient norm, update/weight ratio, and Frobenius weight norm — keyed by parameter name: W_ff.0 (input), W_ff.1 (readout), plus W_ei.1, W_ie.1 only in the init family’s trainable-loop cells (gnorm___ in the jsonl).
test_margin, test_confidence, test_logit_scale	logit-discrimination diagnostics (exp024) — true–runner-up logit margin, softmax confidence in the answer, mean logit .
skipped_steps, new_best, samples	steps skipped on non-finite gradients; new-best flag; samples seen.
elapsed_s, train_compute_s, eval_s, observe_s	cumulative wall time + per-epoch timing breakdown.

Results

1. Canonical reference (θ_u off, all MNIST)

coba and ping at $\theta_u = \text{off}$, all 70k images, seeds 42/43/44 (6 cells) — the full-data baseline. Unconstrained, the two architectures are essentially tied: coba reaches $\approx 95.5\%$ and ping $\approx 94.0\%$, with near-zero across-seed spread. This is the reference point from which the spike-budget sweep pulls them apart.

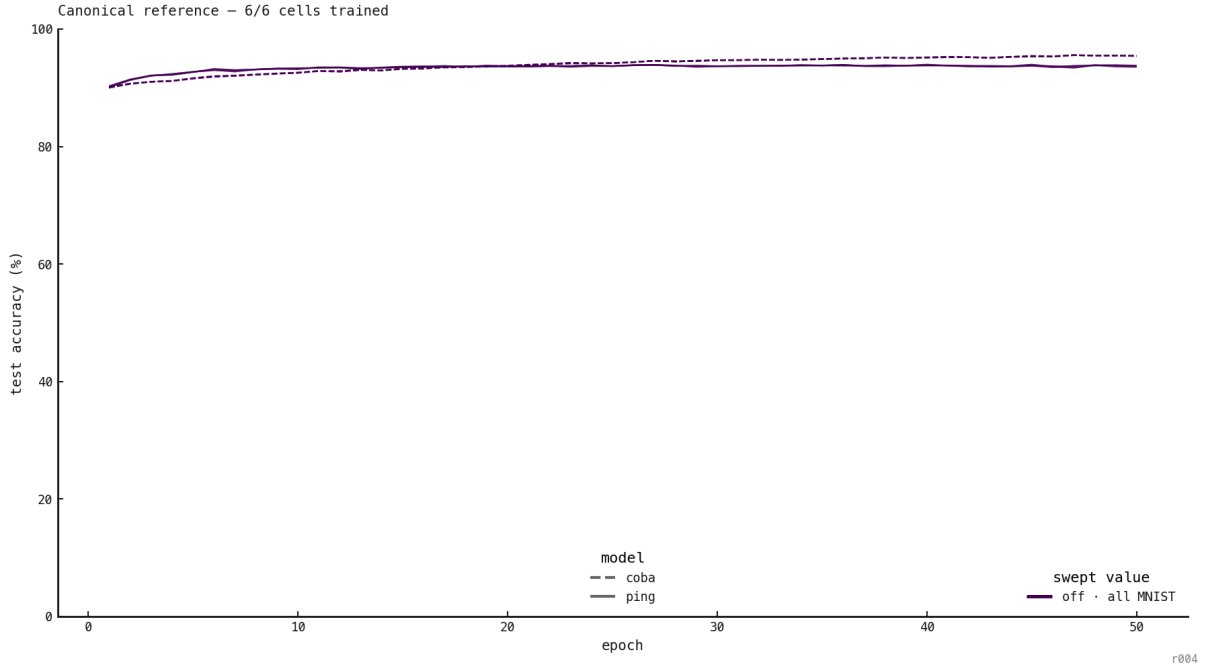


Figure 61: Test accuracy over epochs, canonical full-MNIST cells (coba dashed, ping solid; three seeds each). The two loops reach parity when no spike budget is imposed.

2. θ_u spike-budget sweep

coba and ping across $\theta_u \in \text{off}, 5, 2, 1, 0.5, 0.2$, three seeds each (36 cells) — so the accuracy-rate frontier carries error bars at every point. θ_u is the spike budget in spikes/trial. This is the headline result: as the budget tightens, ping degrades gracefully ($91.6 \rightarrow 86.5\%$) while coba collapses ($90.7 \rightarrow 60.1\%$), a gap that widens monotonically from ≈ 0 to ≈ 26 points. ping’s γ -rhythm gates sparsity; coba’s bare feedforward code cannot pay the budget without shedding accuracy.

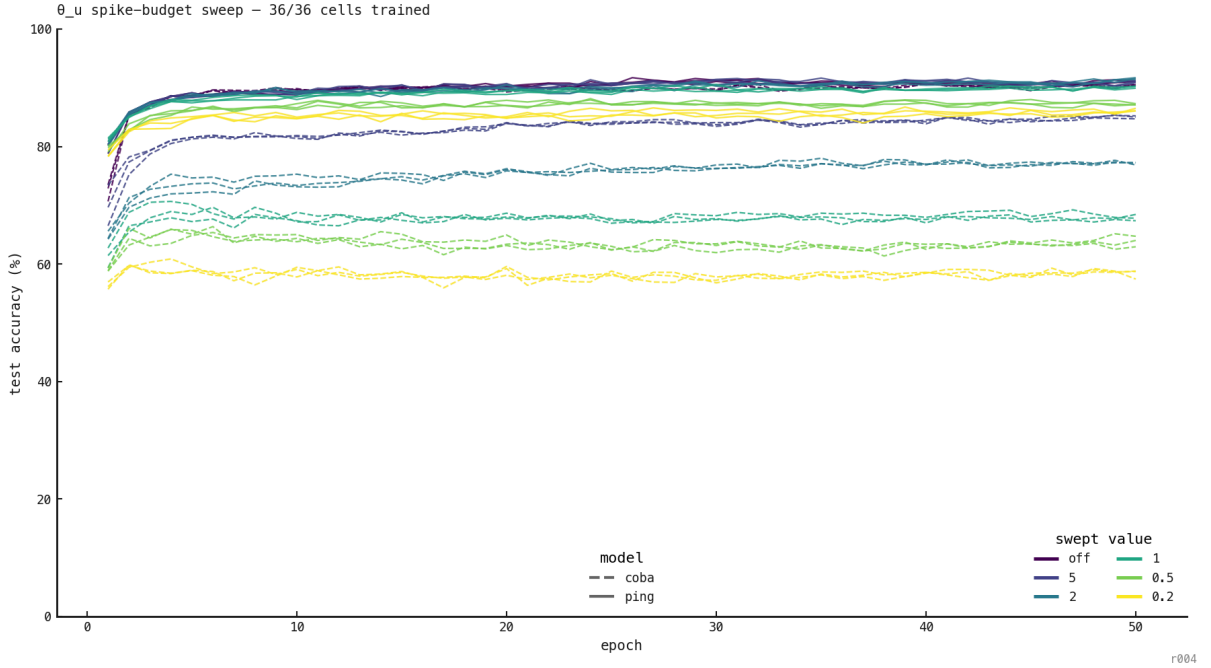


Figure 62: Test accuracy over epochs across the θ_u sweep. Tighter budgets (smaller θ_u) plateau lower — the spike-economy trade-off — and coba falls far faster than ping.

3. τ_{GABA} ladder

ping across $\tau_{\text{GABA}} \in 4.5, 6, 9, 12, 18, 27$ ms, three seeds each (18 cells). Accuracy is largely insensitive to inhibitory decay ($\approx 88\text{--}92\%$ across the ladder), but the **rhythm** is not: measured from the trained networks, the γ -frequency falls monotonically with τ_{GABA} (≈ 50 Hz at 4.5 ms $\rightarrow \approx 19$ Hz at 27 ms), sitting at ≈ 45 Hz at the canonical $\tau_{\text{GABA}} = 6$ ms — matching the operating point (Appendix).

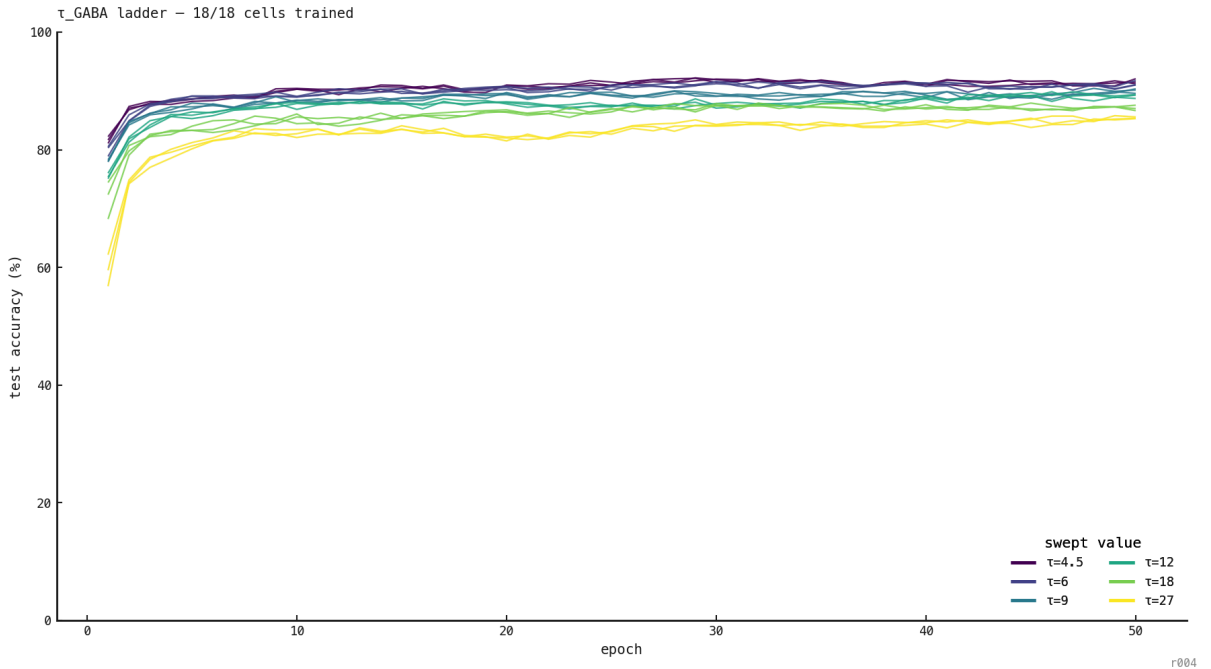


Figure 63: Test accuracy over epochs across the τ_{GABA} ladder; cells converge to similar accuracy regardless of inhibitory decay.

4. Δt sweep

ping across $\Delta t \in 0.05, 0.1, 0.25, 0.5, 1.0$ ms (physical T fixed), three seeds each (15 cells) — the documented timestep exception. Accuracy is flat across the sweep ($\approx 90.4\text{--}91.4\%$): the integrator is robust to timestep from 0.1 to 1.0 ms, and the 0.05 ms cells (which need ≈ 31 GB and so ran on a 5090) agree.

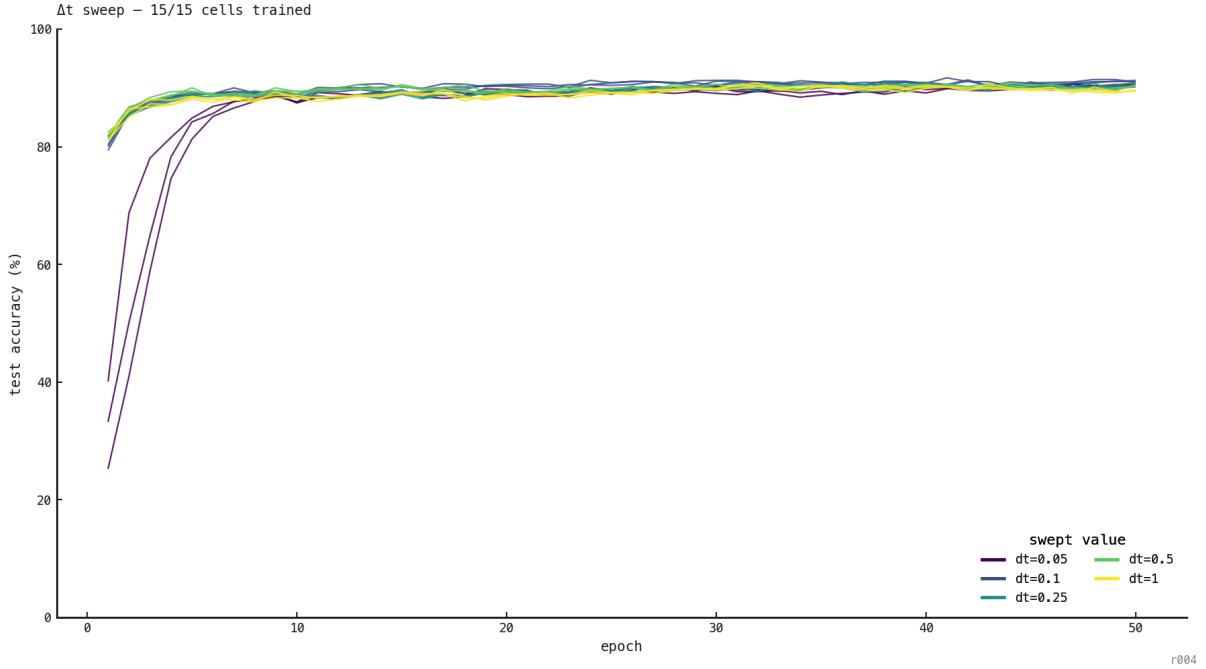


Figure 64: Test accuracy over epochs across the integration-timestep sweep; accuracy is insensitive to Δt over the tested range.

5. Init variants

ping with four recurrent-loop inits — frozen PING, trainable from PING / zero / small seed — three seeds each (12 cells). All reach $\approx 89\text{--}91\%$, but only the frozen-PING control keeps the true E/I regime ($E \approx 10$ Hz, $I \approx 62$ Hz): the trainable-loop cells drift toward a feedforward code (high E, low or zero I) — the zero-init cells never engage inhibition at all ($I \approx 0$ Hz). Comparable accuracy, but the rhythm is not learned when it is not built in.

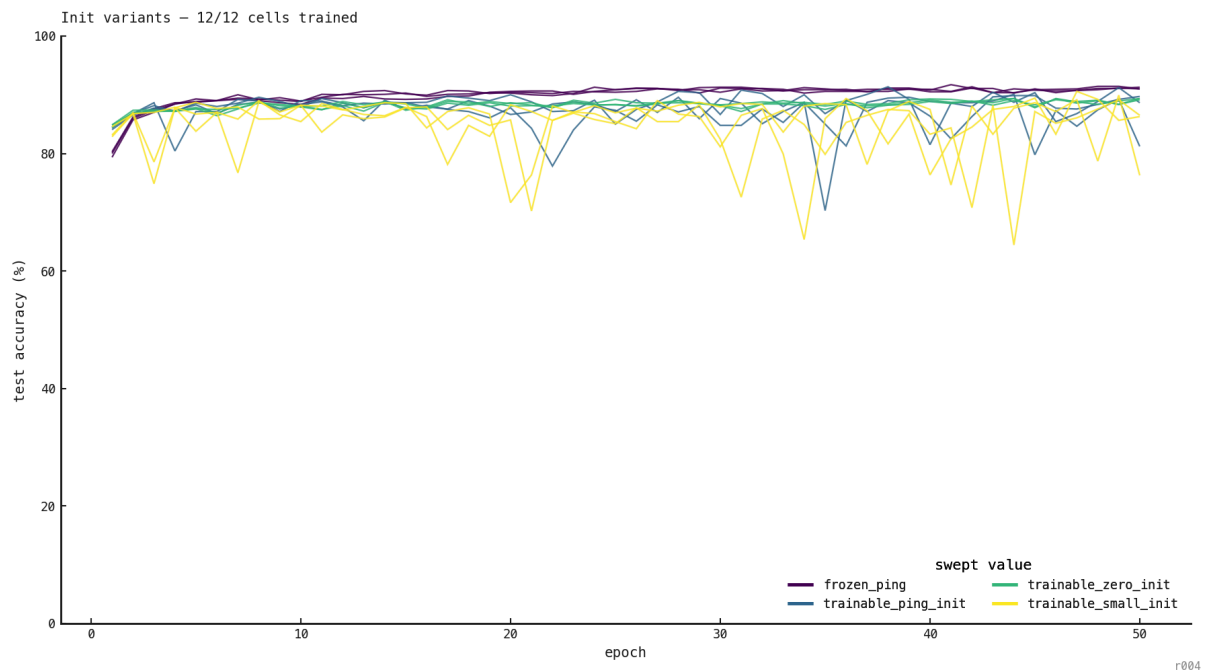


Figure 65: Test accuracy over epochs across the recurrent-loop inits; trainable-loop cells learn noisier curves than the frozen control.

Appendix — per-config spike rasters (digit 0)

The same fixed MNIST image (digit 0, sample 0) sent through each trained network (seed 42 as the per-config representative), so every raster is directly comparable. E cells sit below the divider, I cells above; the lower panel is the 1 ms population rate. These are the visual counterpart to the firing-rate and rhythm numbers above — sparse γ -rhythmic PING versus dense asynchronous COBA, and how each regime deforms under the sweeps.

A.1 Canonical reference

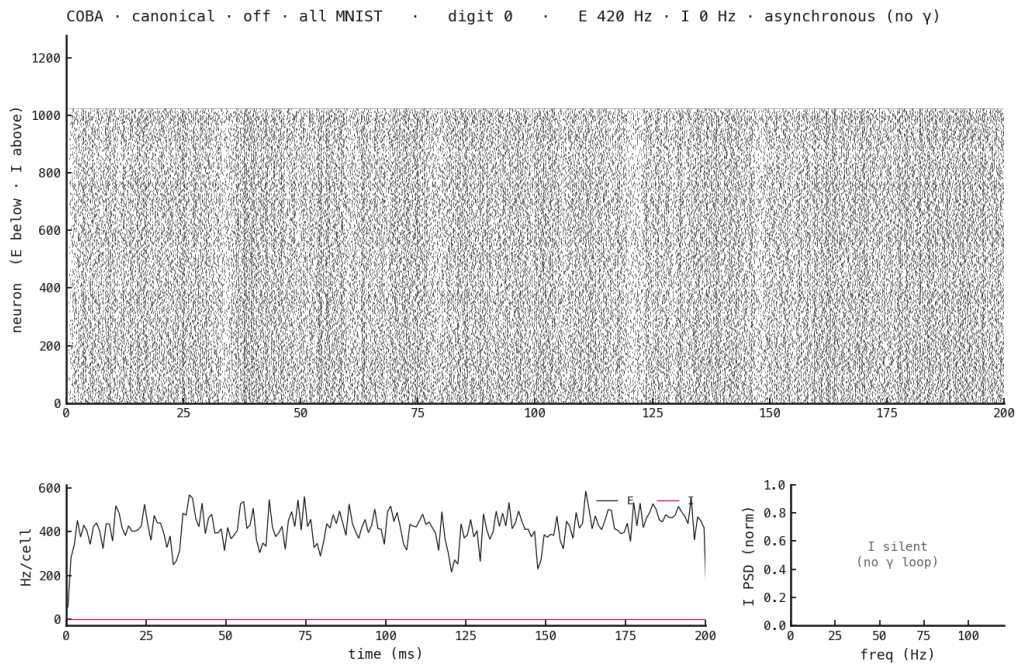


Figure 66: COBA, canonical = off · all MNIST.

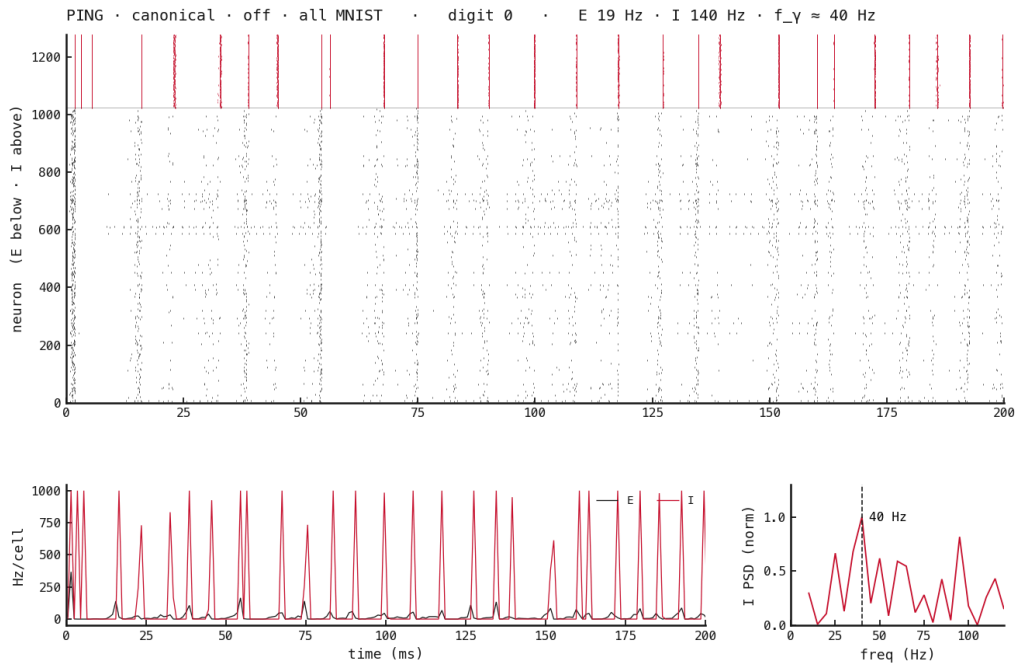


Figure 67: PING, canonical = off · all MNIST.

A.2 θ_u spike-budget sweep

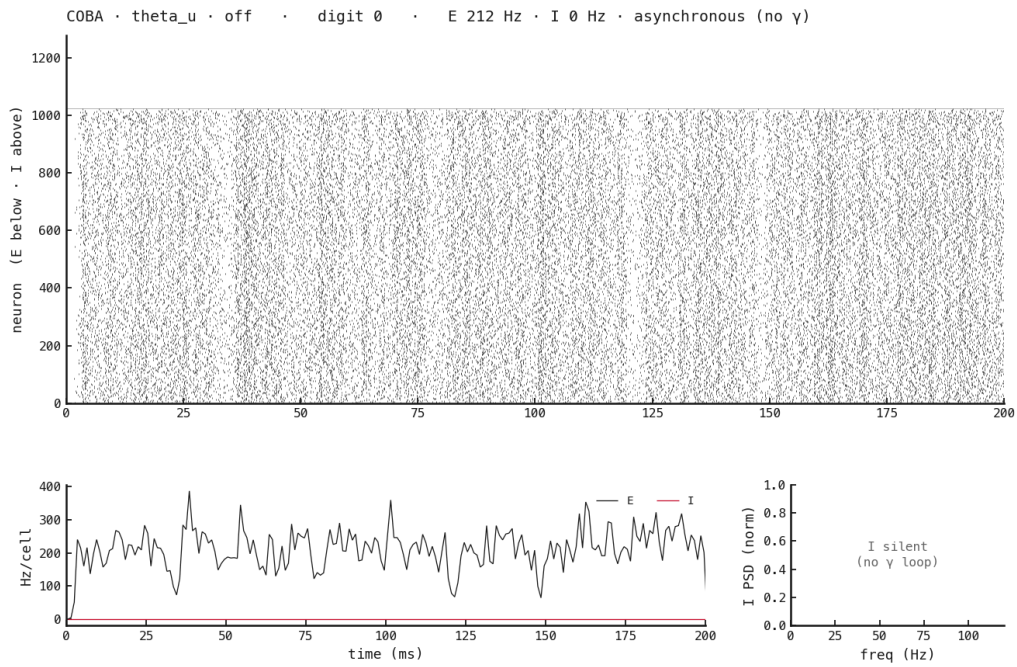


Figure 68: COBA, $\theta_u = \text{off}$.

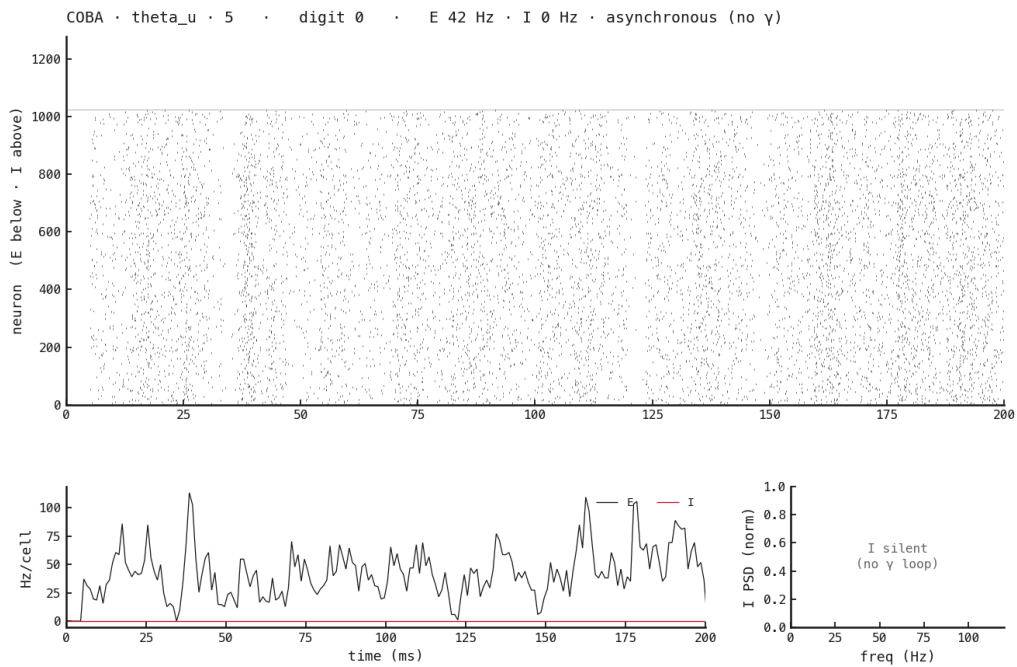


Figure 69: COBA, $\theta_u = 5$.

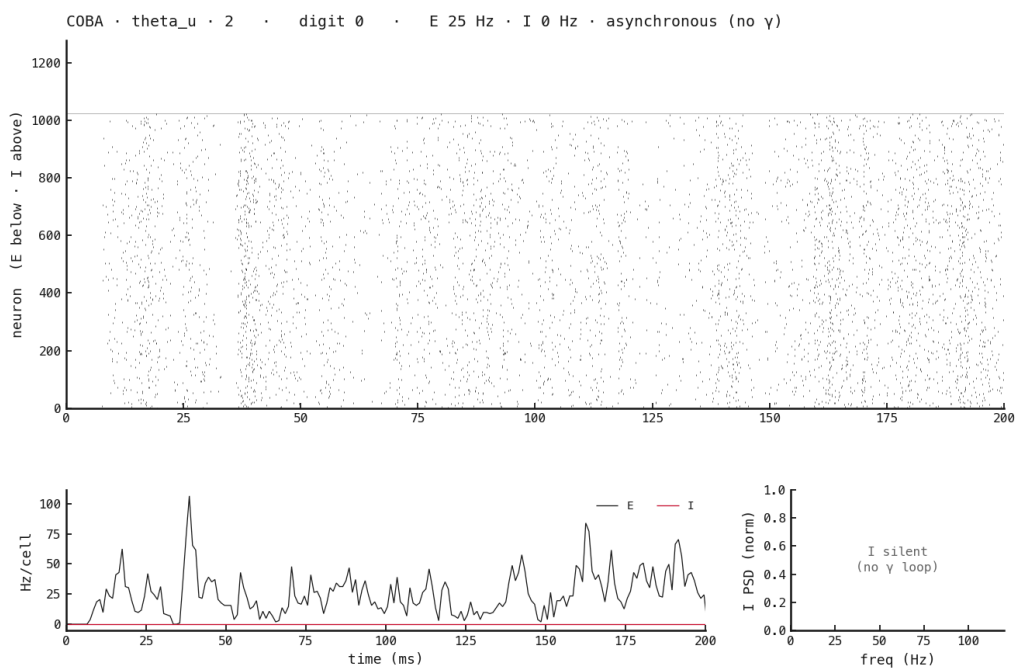


Figure 70: COBA, theta $u = 2$.

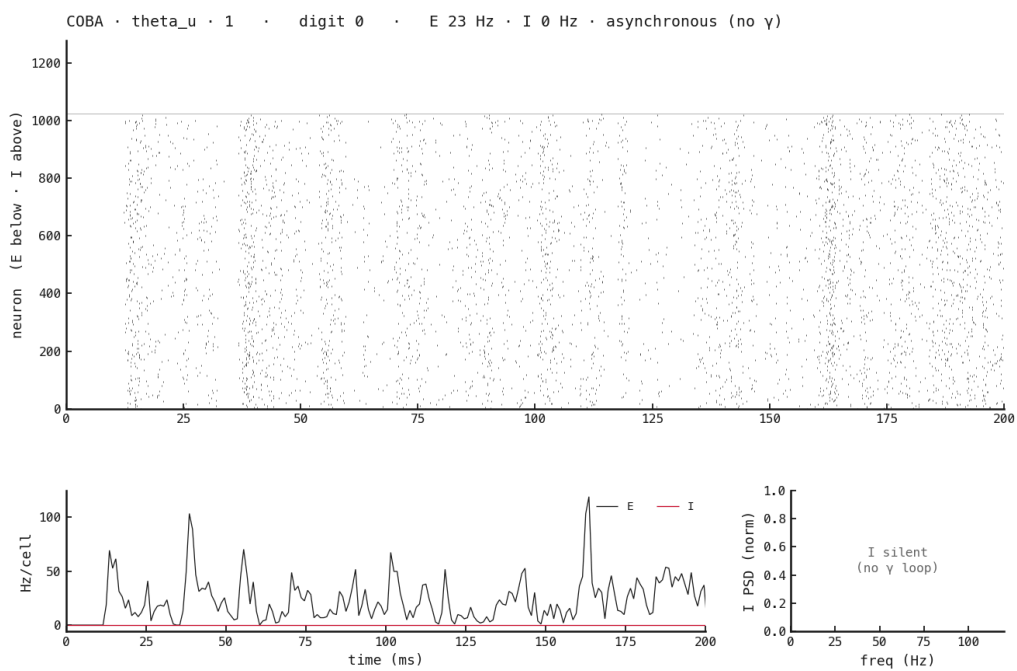


Figure 71: COBA, theta $u = 1$.

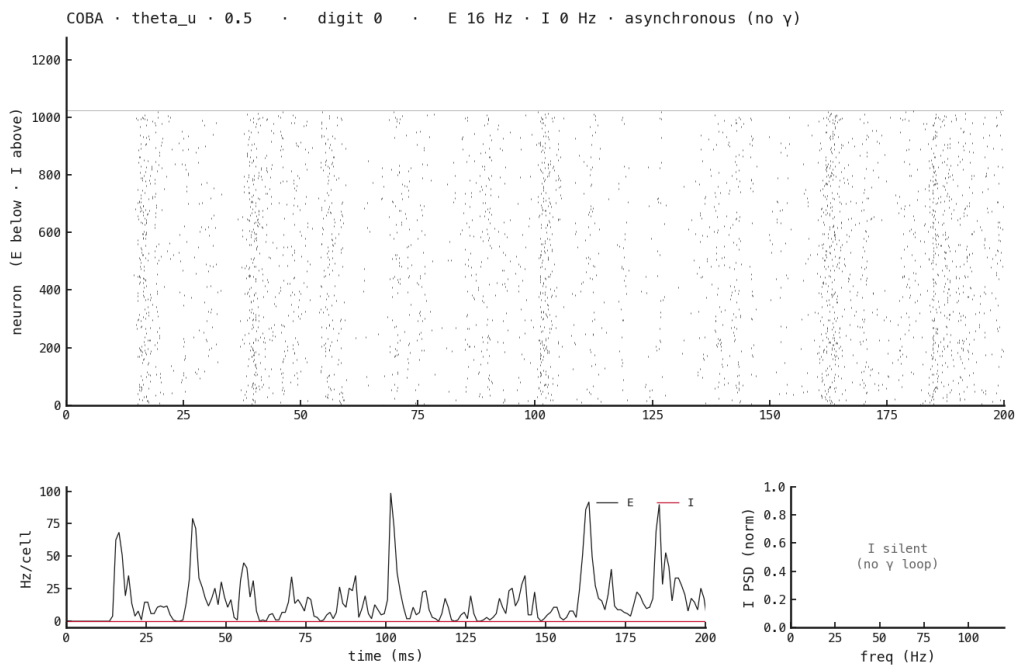


Figure 72: COBA, theta $u = 0.5$.

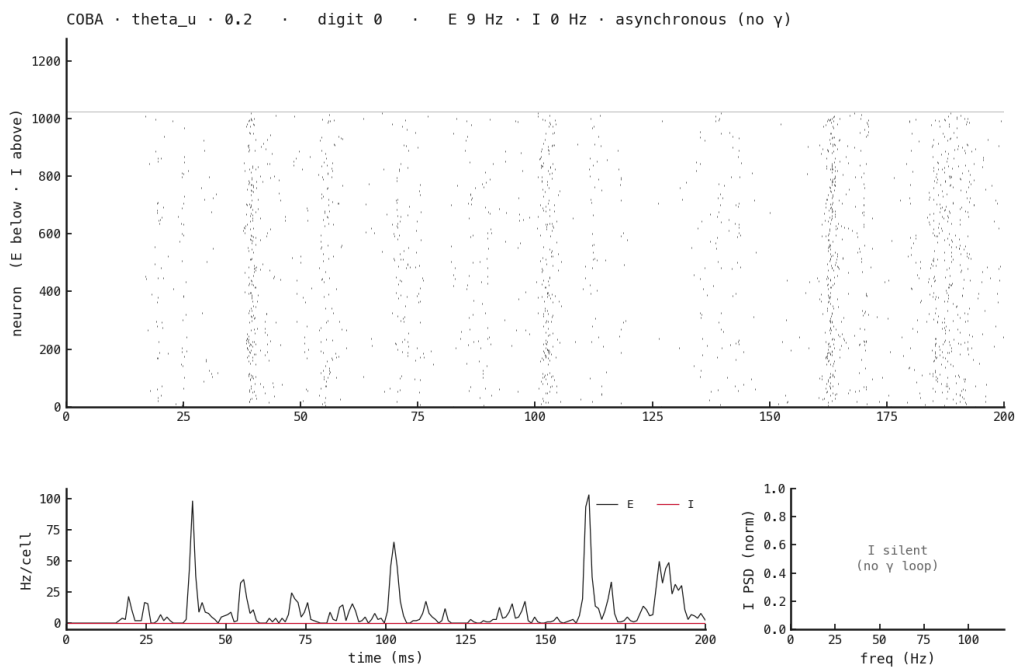


Figure 73: COBA, theta $u = 0.2$.

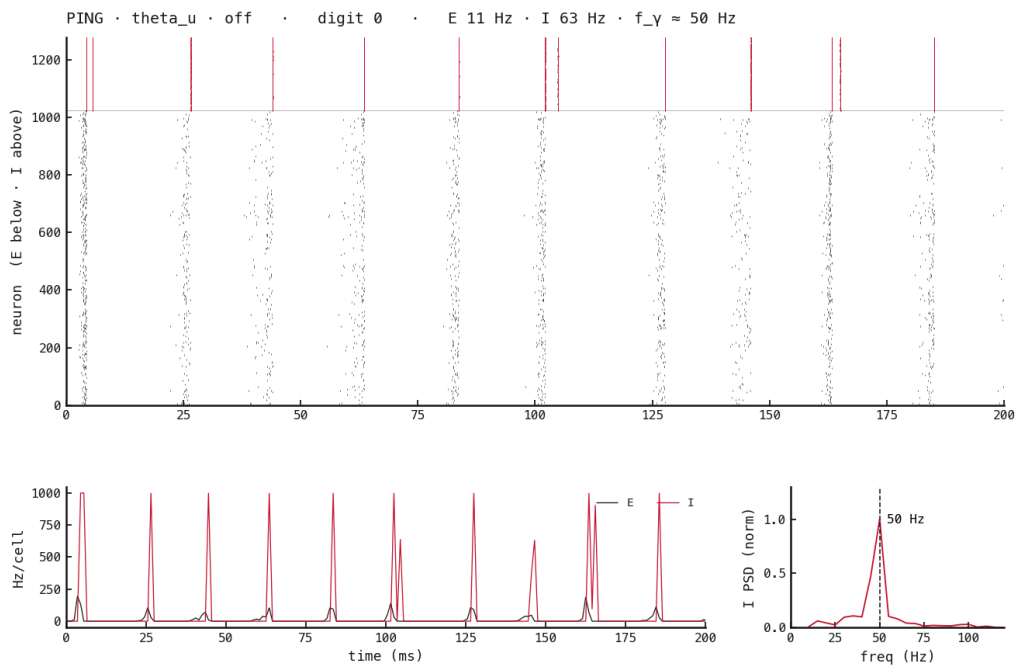


Figure 74: PING, theta u = off.

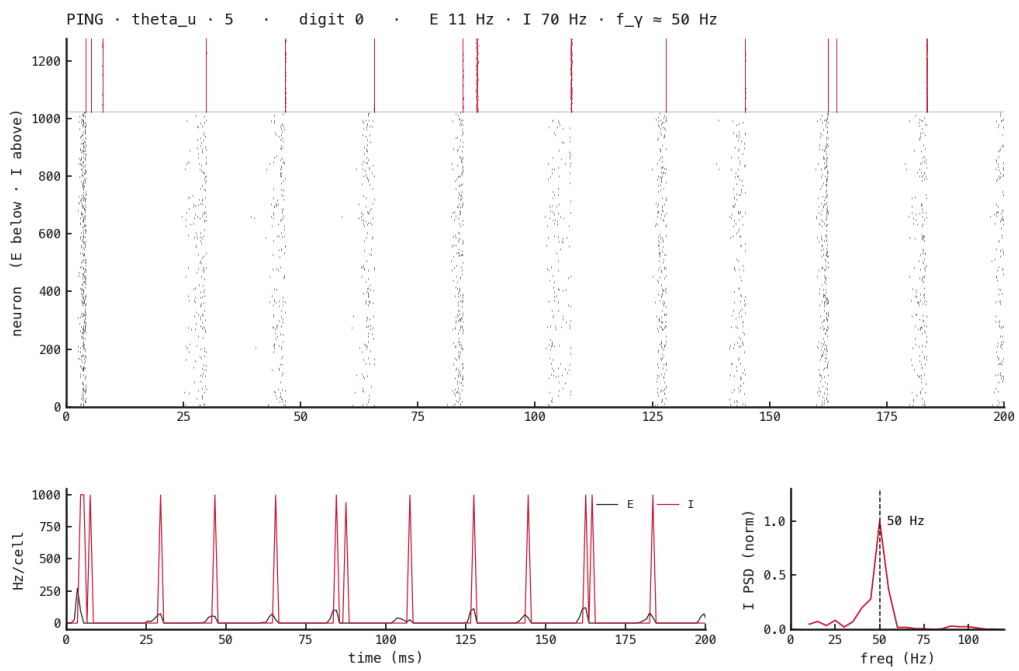


Figure 75: PING, theta u = 5.

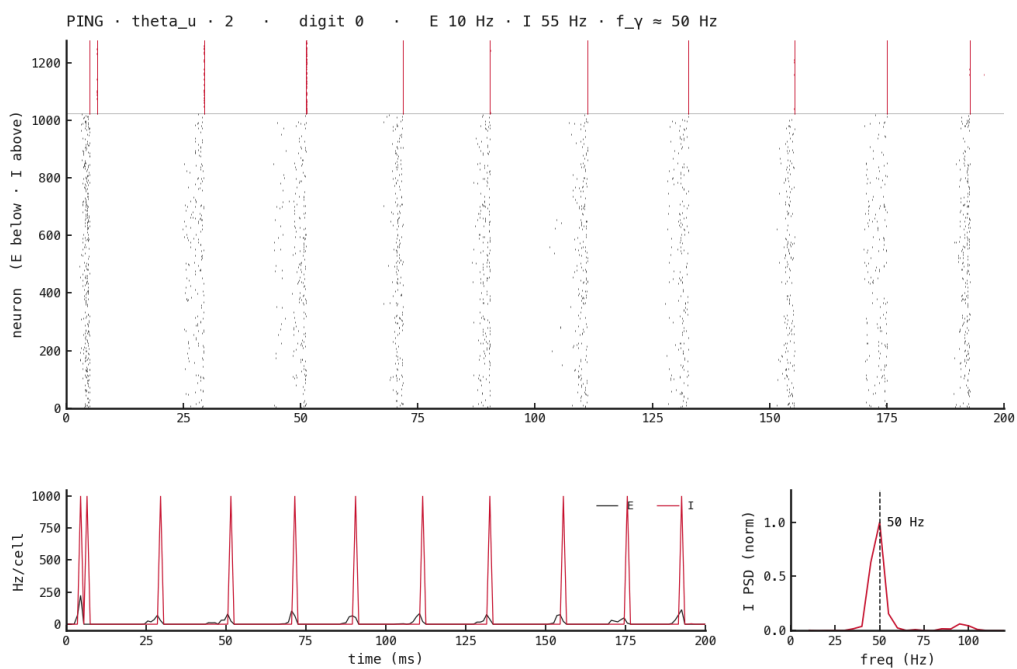


Figure 76: PING, theta $u = 2$.

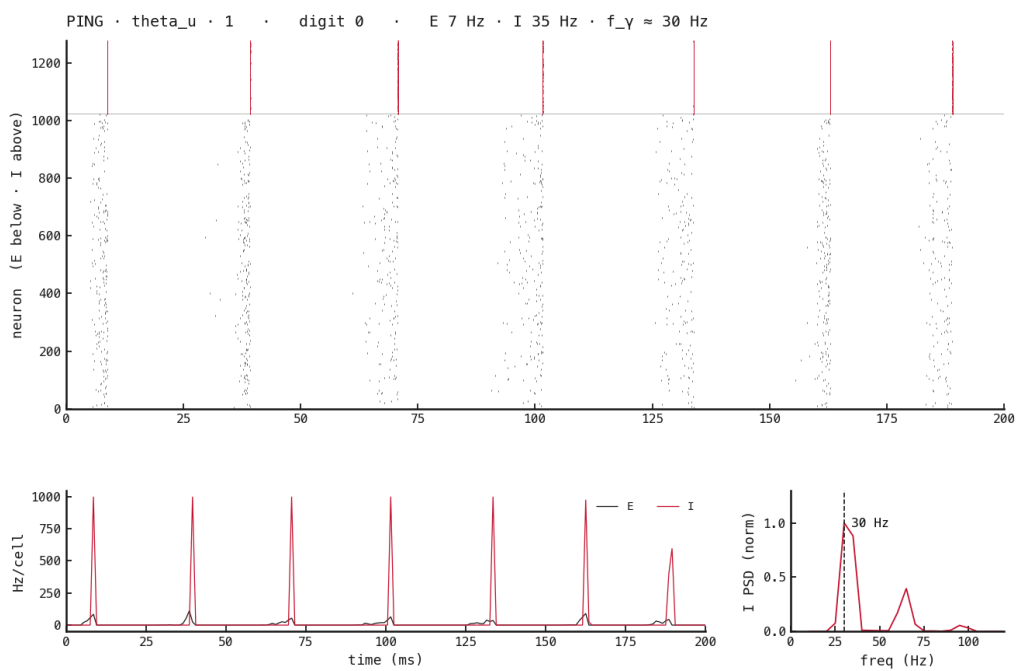


Figure 77: PING, theta $u = 1$.

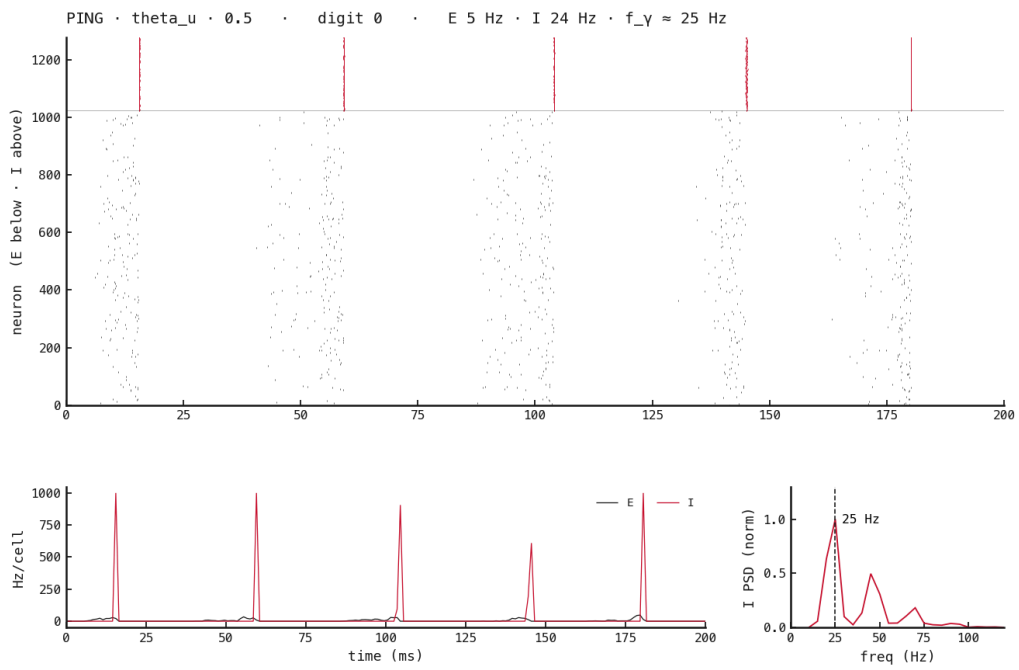


Figure 78: PING, theta u = 0.5.

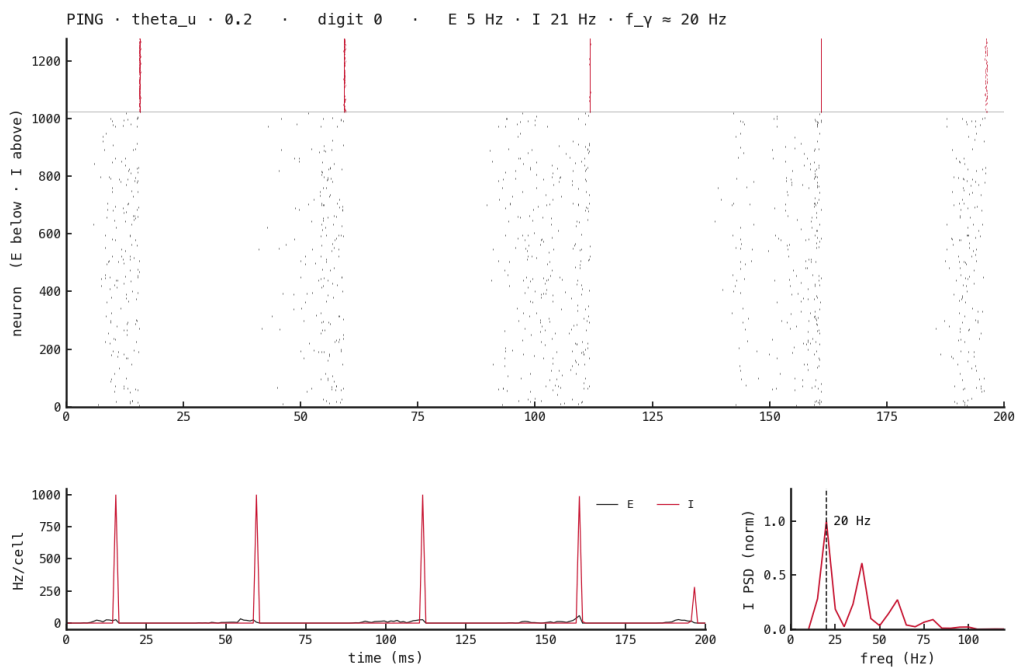


Figure 79: PING, theta u = 0.2.

A.3 τ_{GABA} ladder

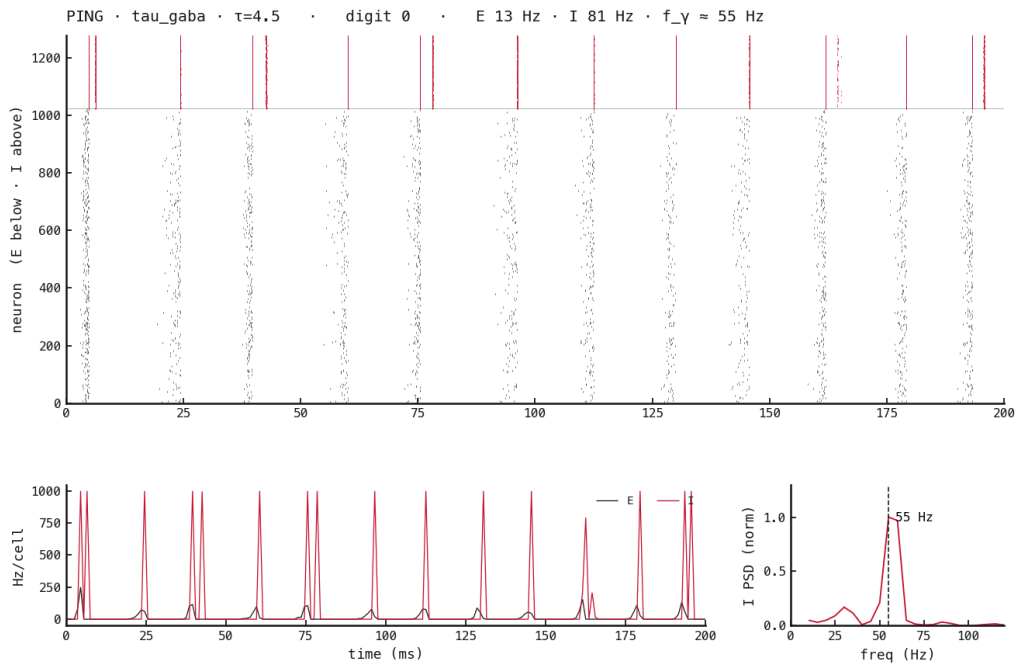


Figure 80: PING, tau gaba = $\tau=4.5$.

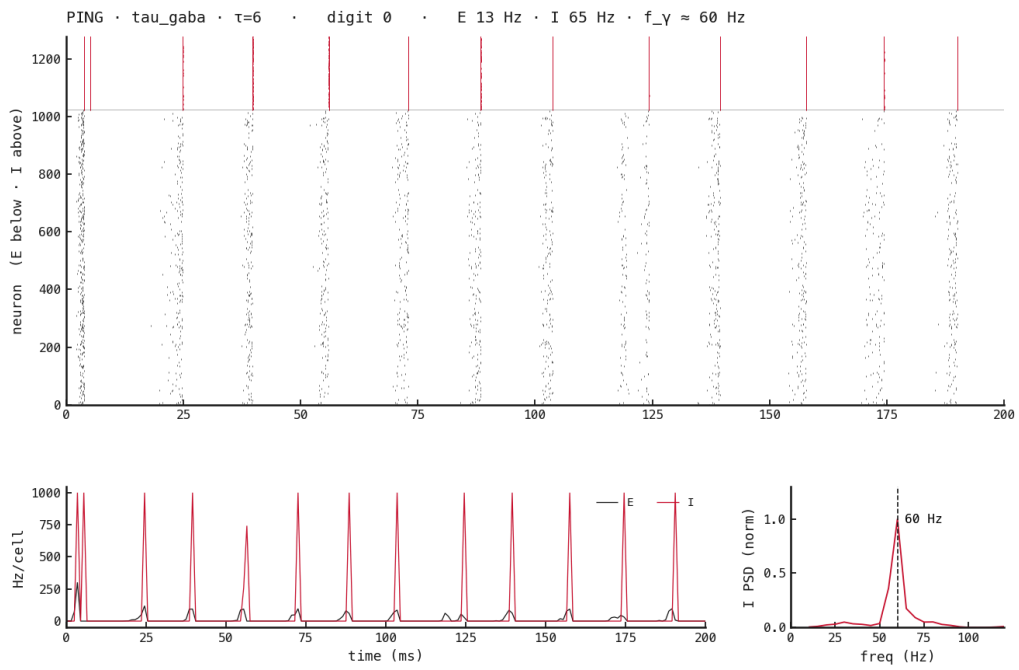


Figure 81: PING, tau gaba = $\tau=6$.

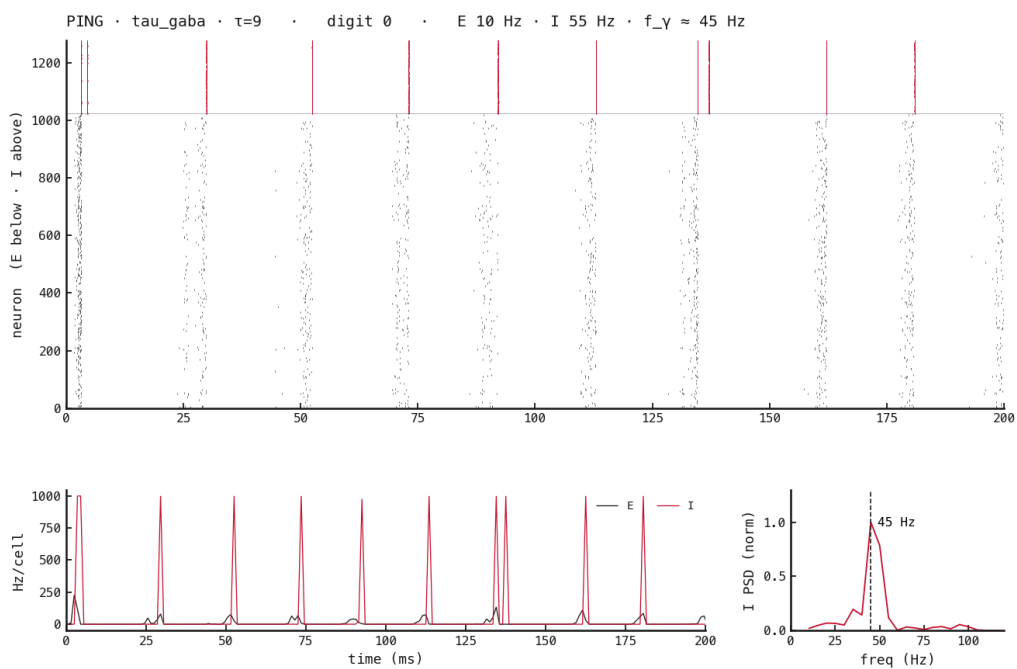


Figure 82: PING, tau gaba = $\tau=9$.

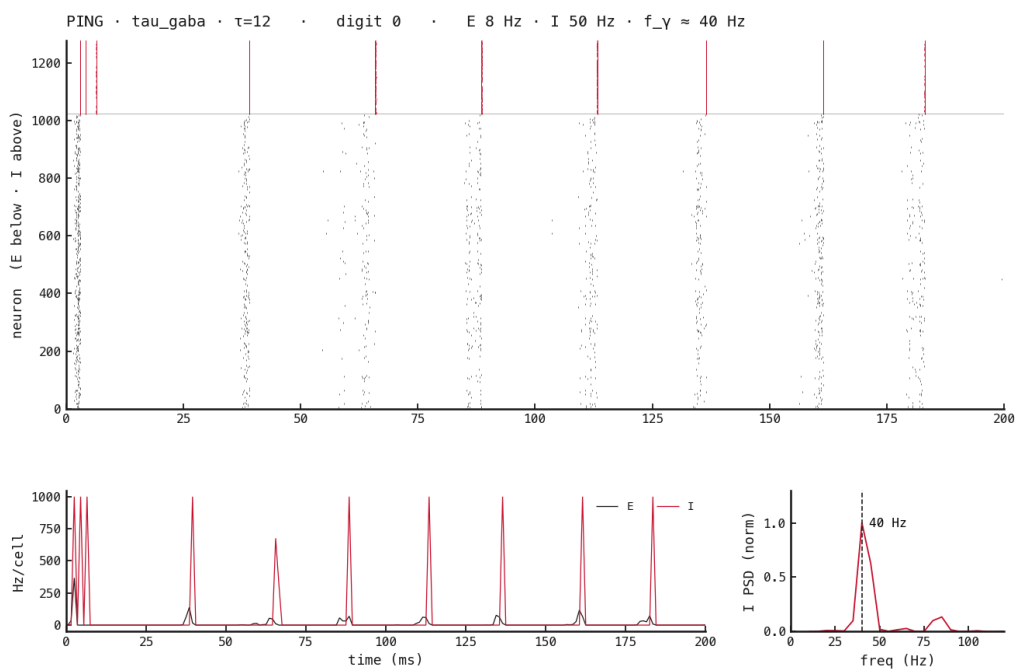


Figure 83: PING, tau gaba = $\tau=12$.

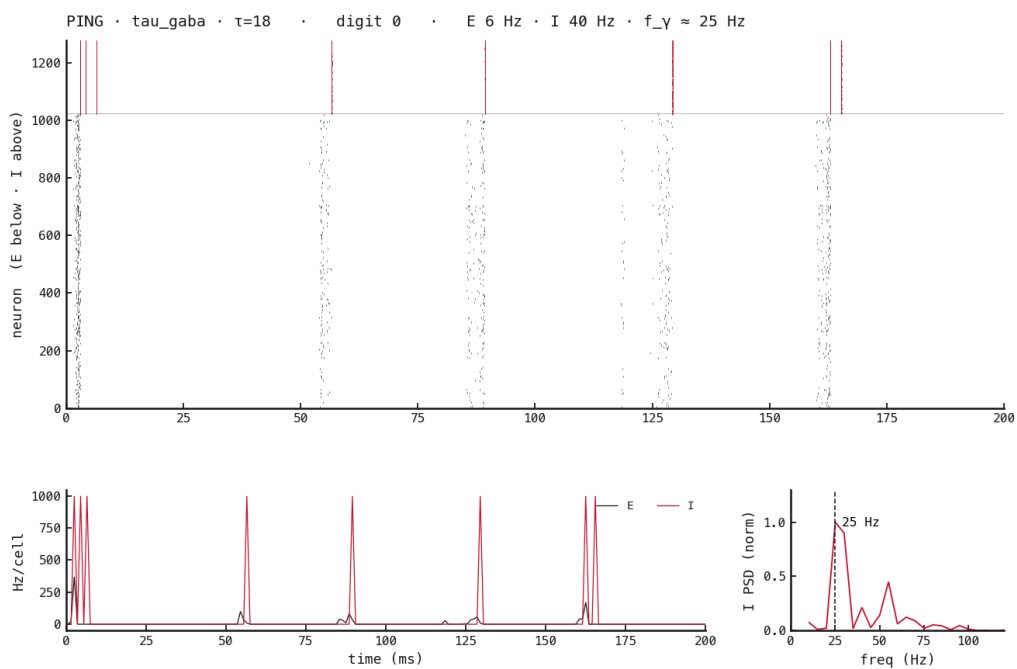


Figure 84: PING, tau gaba = $\tau=18$.

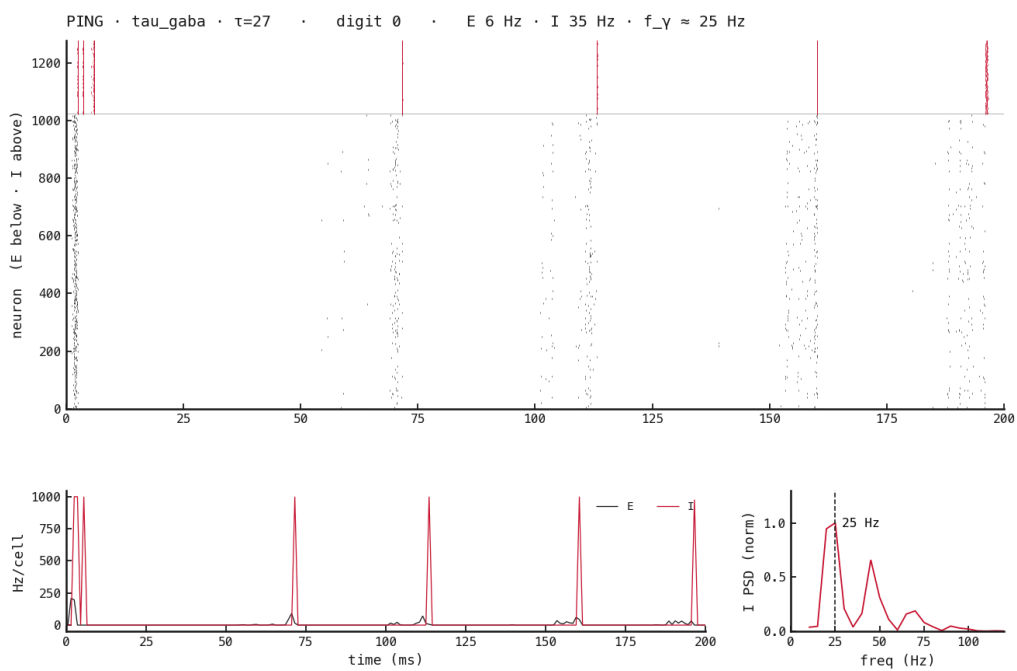


Figure 85: PING, tau gaba = $\tau=27$.

A.4 Δt sweep

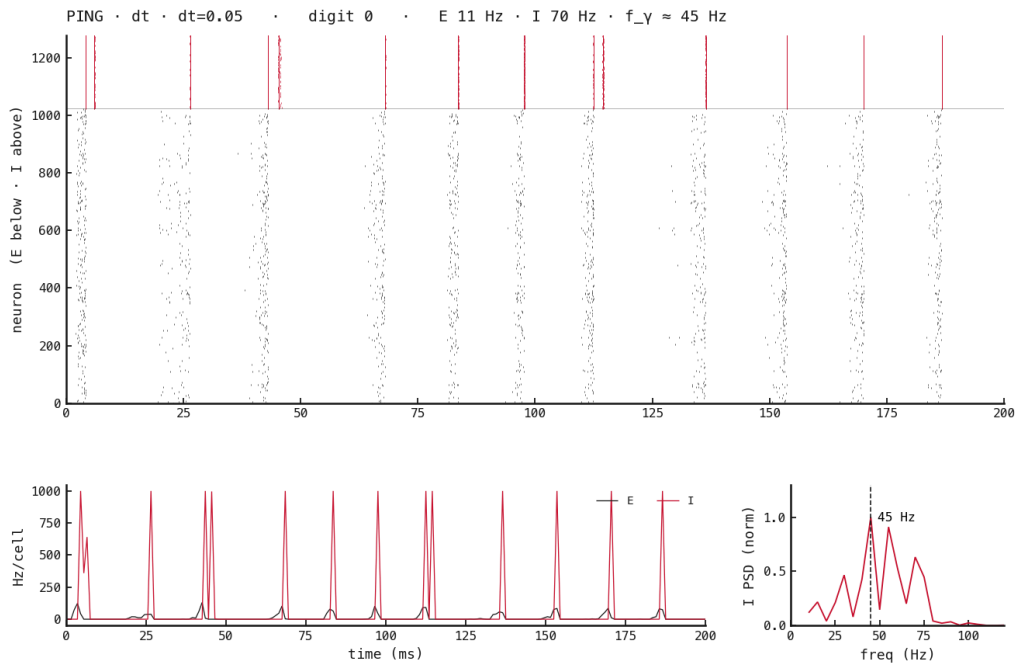


Figure 86: PING, $dt = dt=0.05$.

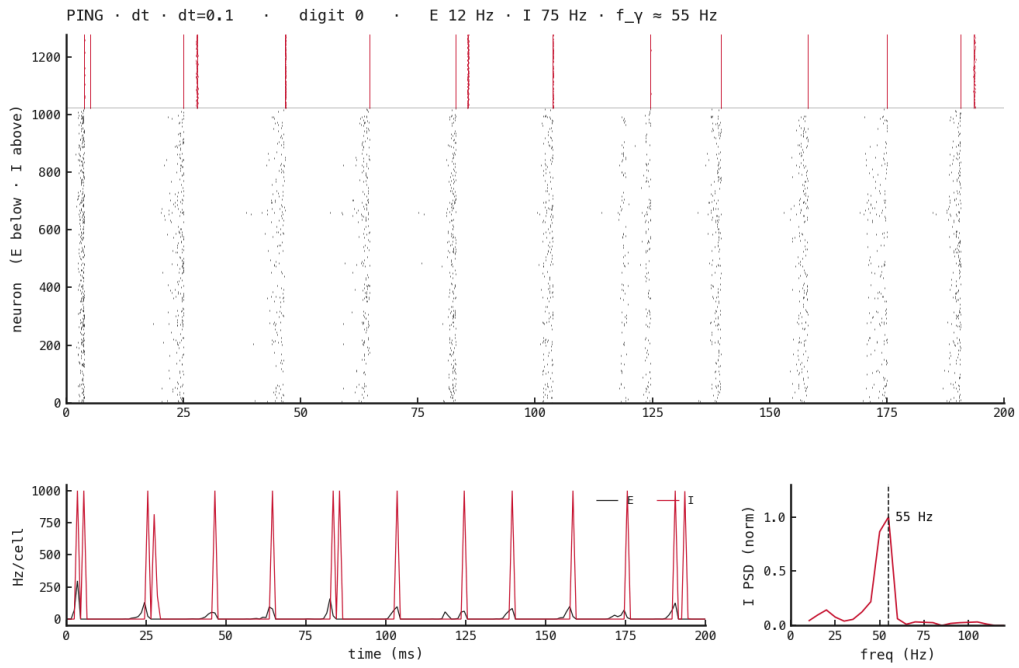


Figure 87: PING, $dt = dt=0.1$.

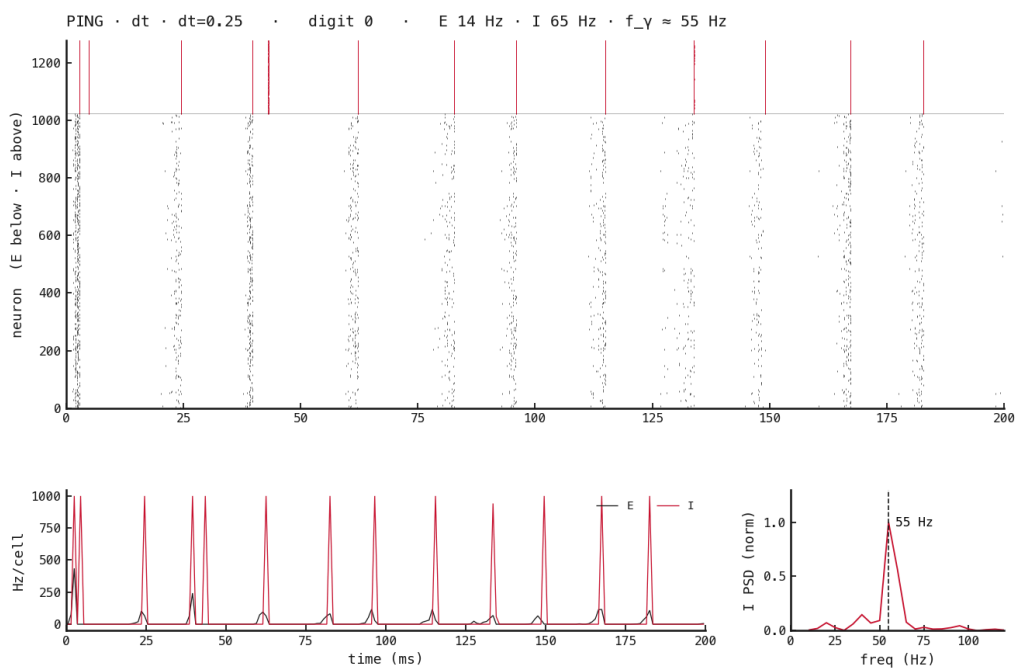


Figure 88: PING, $dt = dt=0.25$.

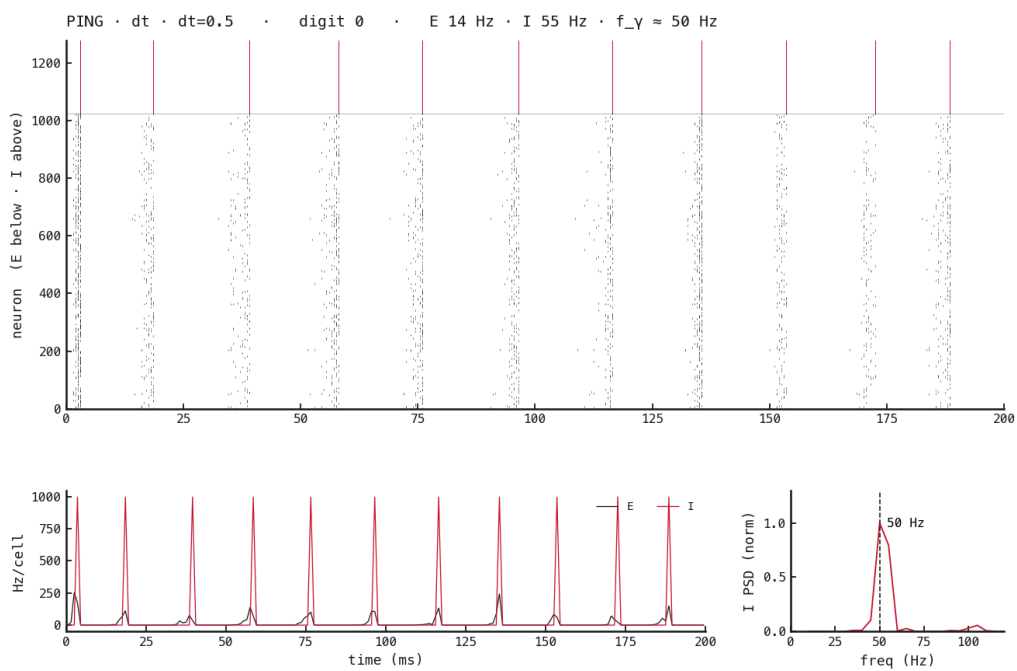


Figure 89: PING, $dt = dt=0.5$.

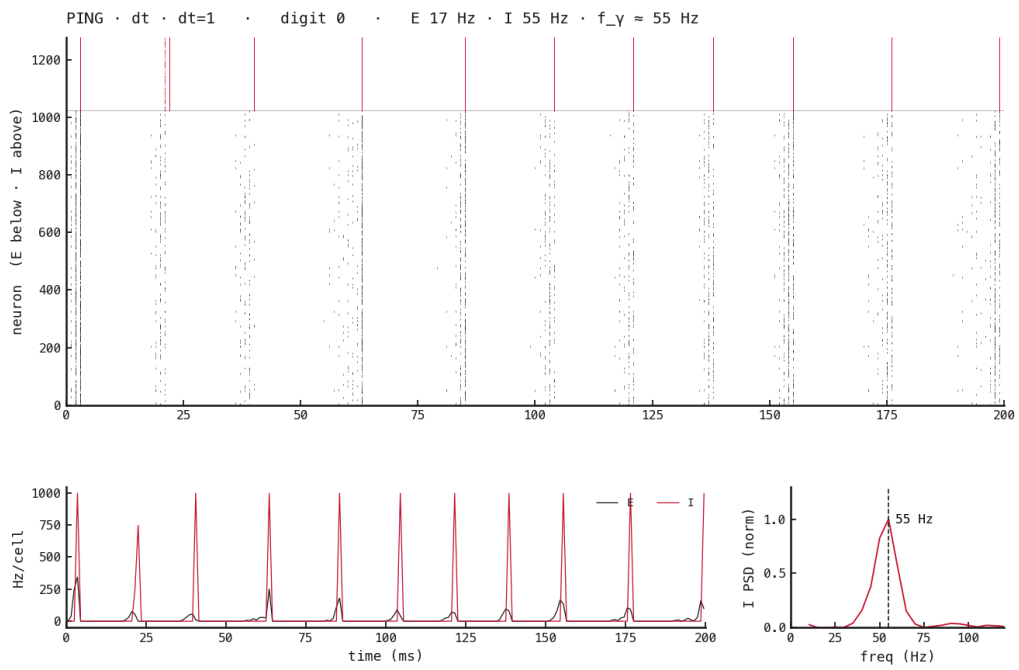


Figure 90: PING, $dt = dt=1$.

A.5 Init variants

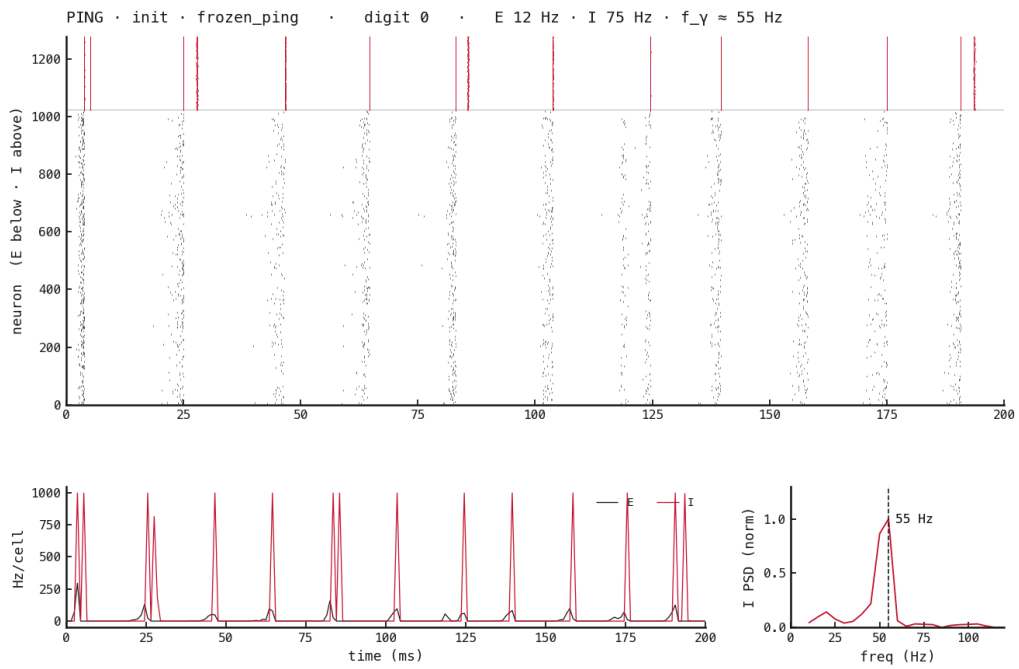


Figure 91: PING, $init = frozen_ping$.

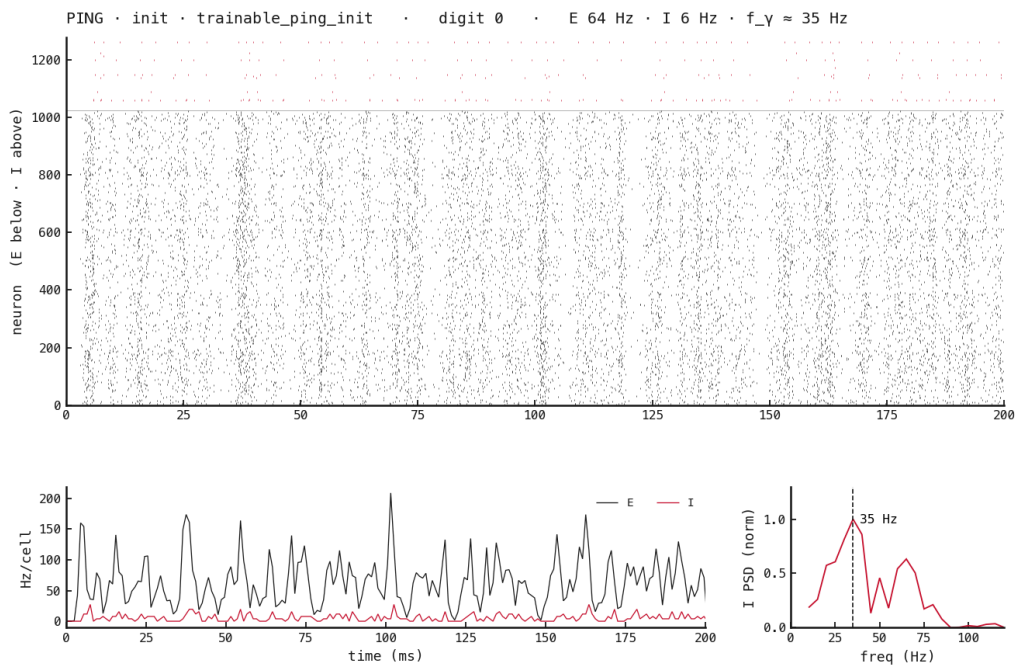


Figure 92: PING, init = trainable_ping_init.

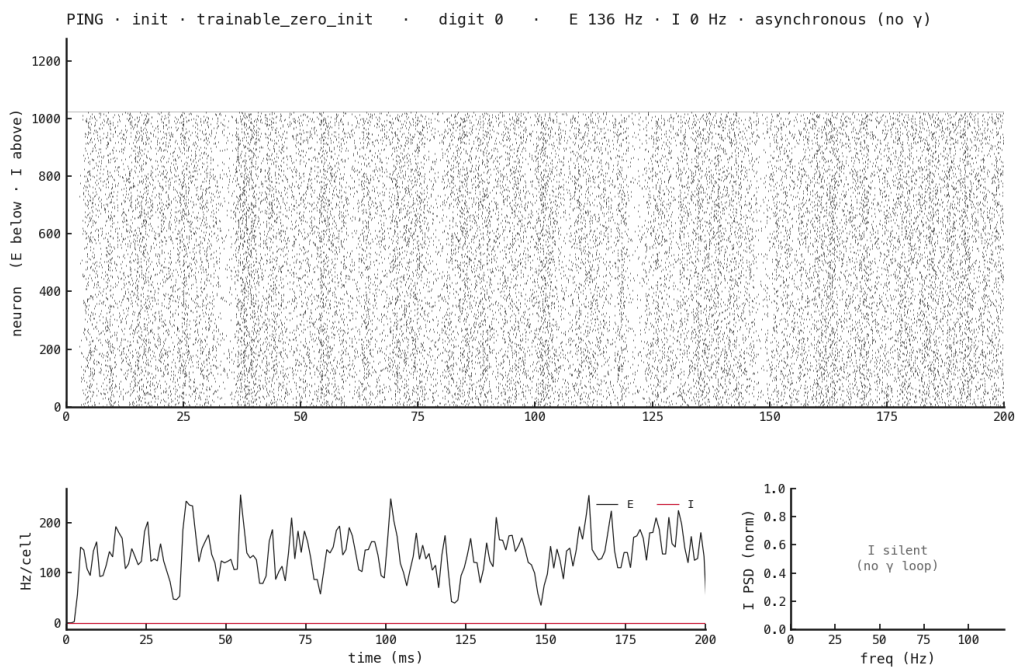


Figure 93: PING, init = trainable_zero_init.

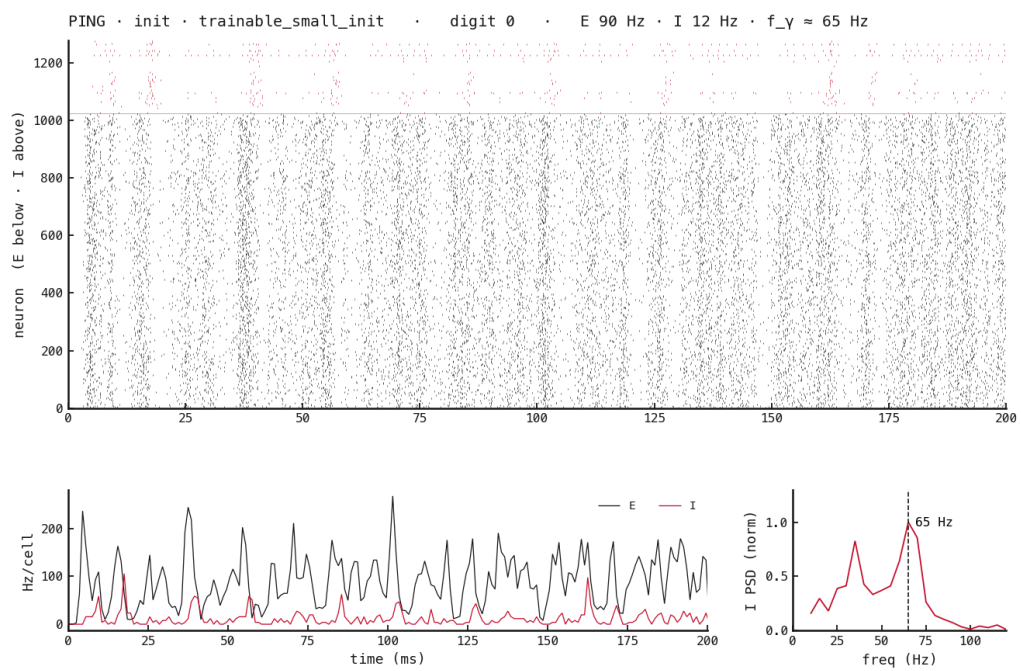


Figure 94: PING, init = trainable_small_init.